Cloud Computing

Service Models and Economics

Lecture 02 - Part 2

Academic Year 2025/2026

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste

Recap: Part 1

What we covered:

- NIST definition and 5 essential characteristics
- Service models: IaaS → PaaS → SaaS (the stack)
- Deployment models: Public, Private, Hybrid, Multi-cloud

Part 2 covers:

FaaS/Serverless → Cloud Economics → Pricing Models → TCO → FinOps → Providers

Part 4

FaaS: Function as a Service

The serverless model

FaaS: Function as a Service

Provider manages: Everything including scaling and execution
You manage: Individual functions (code) only

Key characteristics:

- Event-driven: Functions triggered by events (HTTP, queue, schedule)
- Stateless: No persistent state between invocations
- Auto-scaling: Scales to zero when idle, scales up with demand
- Sub-second billing: Pay per invocation and execution time

Examples: AWS Lambda, Azure Functions, Google Cloud Functions

FaaS: A Different Model

Traditional Server

- Always running (pay 24/7)
- You manage capacity
- Idle time = wasted money
- Full application deployed

FaaS / Serverless

- Runs on demand only
- Auto-scales to load
- Pay only when running
- Individual functions

"Serverless" doesn't mean no servers – it means you don't manage them

When to Use FaaS

Good fit:

- Event processing (S3 uploads, queues)
- APIs with variable traffic
- Scheduled tasks (cron jobs)
- Webhooks and integrations

Poor fit:

- Long-running processes (>15 min)
- Stateful applications
- Consistent high traffic (always on)
- Low-latency requirements

Cold start: First invocation may be slower (container startup)

Complete Service Model Stack

SaaS	Gmail, Salesforce	You control: Data & config
FaaS	Lambda, Cloud Functions	You control: Code (functions)
PaaS	Heroku, App Engine	You control: App & data
IaaS	EC2, Azure VMs	You control: OS and up

↑ Less control, less management ↓ More control, more management

Part 5

Cloud Architecture and Concepts

The financial model behind cloud

BASIC CONCEPTS

Abstraction:

Cloud computing **abstracts the details of system implementation from users and developers.** Applications run on physical systems that aren't specified, data is stored in locations that are unknown, administration of systems is outsourced to others, and access by users is ubiquitous.

Virtualization:

Cloud computing **virtualizes systems by pooling and sharing resources.** Systems and storage can be provisioned as needed from a centralized infrastructure, costs are assessed on a metered basis, multi-tenancy is enabled, and resources are scalable with agility.

Two main Concept to Remember when using a Cloud

Cloud Architecture

The advances in system virtualization is mixed with standard networking protocols already used in Internet.

The cloud creates a system where resources can be pooled and partitioned as needed.

Cloud architecture can couple software running on virtualized hardware in multiple locations to provide an on- demand service to user-facing hardware and software

Virtualized resources are easy to optimize and modify

A Cloud is a composable system based on Internet Standards and protocols

Composability

Each component used during your development process is pluggable and can be replaced, scaled, and consistently improved to help you meet your business needs. When establishing composable architecture, each component of your development stack and **microservices** should be able to communicate with one another, regardless of differences in language or code.

Modular:

- It is a self-contained and independent unit that is cooperative, reusable, and replaceable.

Stateless:

- A transaction is executed without regard to other transactions or requests.

What is Microservice Architecture?

See also FaaS…

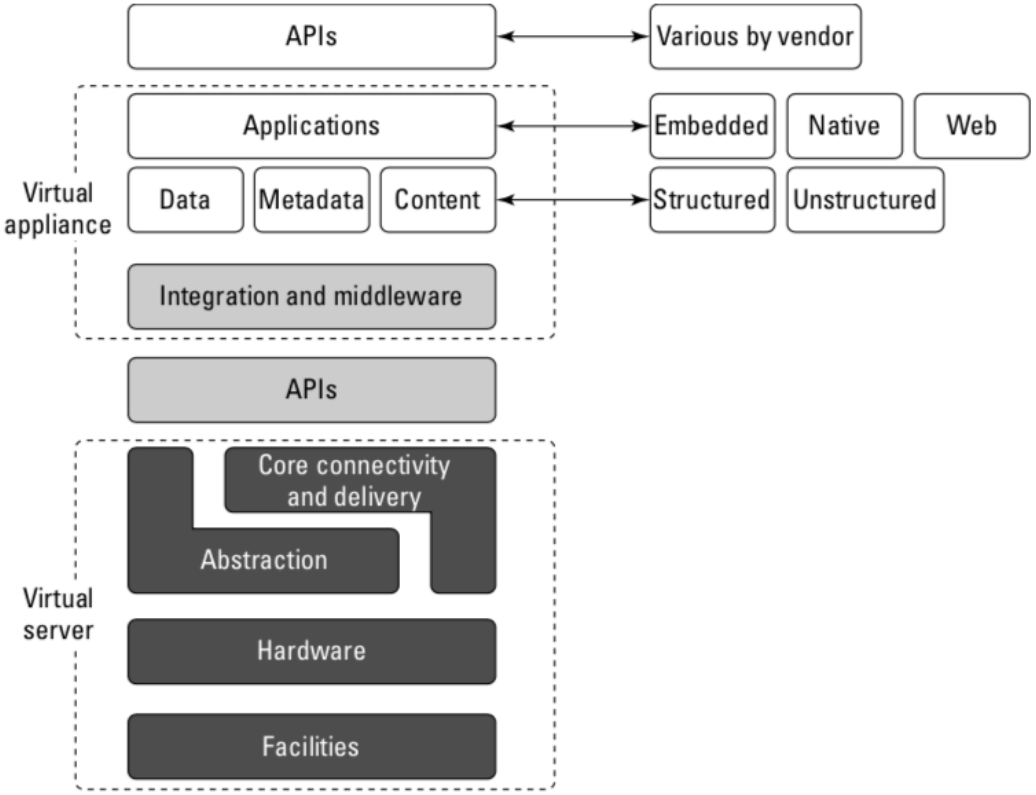
Isolate tasks	a task can range from just a simple function (i.e. serve a file to download) to complex computer programs (i.e. classify images using a neural network).
Standard Access	Microservices are interacted with using a well-defined interface, usually a REST API over HTTP, but a Secure Shell protocol (SSH) is a perfectly viable interface as well
Stateless operations	Any transaction is executed without regard to other transactions or requests

In software engineering, a microservice architecture is an architectural pattern that organizes an application into a collection of loosely coupled, fine-grained services that communicate through lightweight protocols[1]

(1) https://en.wikipedia.org/wiki/Microservices

Microserveces in Cloud

- Isolate tasks
- Standard Access
- Stateless operations



Part 6

Cloud Economics

The financial model behind cloud

CapEx → OpEx

CapEx (Capital Expenditure)

Traditional IT model

- Buy hardware upfront
- Depreciate over 3-5 years
- Fixed cost regardless of usage
- Requires capacity planning
- Large initial investment

OpEx (Operational Expenditure)

Cloud model

- Pay as you go
- Expense immediately
- Variable cost with usage
- Scale on demand
- No upfront investment

Cloud shifts IT from capital investment to operational expense

Why OpEx Matters

Financial benefits of the cloud model:

Cash Flow	No large upfront investment; preserve capital for core business
Flexibility	Scale costs with revenue; reduce when business slows
Risk Reduction	No stranded assets if technology changes or project fails
Tax Treatment	Operating expenses often deductible immediately
Agility	Experiment cheaply; fail fast without sunk costs

This is why startups love cloud: grow infrastructure with the business

Cloud Pricing Models

		Savings
On-Demand Pay per hour/second of use	Variable workloads, testing	**0%**
Reserved Commit for 1-3 years	Steady-state workloads	**30-75%**
Spot/Preemptible Bid on spare capacity	Fault-tolerant, flexible	**60-90%**
Savings Plans Commit to $ spend/hour	Flexible commitments	**20-50%**

Spot Instances: High Savings, High Risk

Up to 90% discount – but instances can be terminated with 2 min notice

Good for:

- Batch processing jobs
- CI/CD pipelines
- Stateless web servers (with LB)
- Big data processing (Spark)

Not for:

- Databases
- Stateful applications
- Long-running single jobs
- Production without fallback

HPC use case: Massively parallel simulations where losing a node is acceptable

Total Cost of Ownership (TCO)

Cloud vs On-Premise: It's not just about compute costs

On-Premise Costs

- Hardware purchase
- Data center / facility
- Power and cooling
- Network infrastructure
- IT staff (24/7)
- Maintenance & support
- Software licenses

Cloud Costs

- Compute (VMs, containers)
- Storage
- Network transfer (egress)
- Managed services
- Support plans
- Training & skills
- (Often forgotten: egress!)

Honest TCO comparison requires including ALL costs on both sides

Hidden Cloud Costs

What catches people by surprise

- ⚠ **Data Egress:** Transferring data OUT of cloud is expensive ($0.09/GB+)
- ⚠ **Cross-AZ Traffic:** Traffic between availability zones is charged
- ⚠ **API Calls:** S3 GET/PUT requests, Lambda invocations add up
- ⚠ **Idle Resources:** Forgotten instances, unattached volumes, old snapshots
- ⚠ **Over-provisioning:** Running larger instances than needed
- ⚠ **Premium Support:** Enterprise support plans can be 3-10% of spend

Rule of thumb: Budget 20-30% above computed estimates for surprises

Part 7

Introduction to FinOps

Cloud financial management

What is FinOps?

FinOps = Finance + DevOps
An operational framework for cloud financial management

Core principles:

- Teams need to own their cloud costs (accountability)
- Decisions are driven by business value, not just cost
- Everyone takes ownership (finance, engineering, business)
- Reports should be accessible and timely
- A centralized team drives FinOps practices

FinOps Practices

Tagging	Label all resources with owner, project, environment
Budgets & Alerts	Set spending limits, alert before exceeded
Rightsizing	Match instance size to actual usage
Reserved Capacity	Commit to predictable workloads for discounts
Cleanup Automation	Auto-delete unused resources
Cost Allocation	Chargeback/showback to teams

FinOps is a continuous process, not a one-time project

Part 7

Major Cloud Providers

AWS, Azure, GCP comparison

The Big Three Cloud Providers

AWS	~32% market	Broadest service catalog, most mature
Azure	~23% market	Enterprise integration, hybrid cloud
GCP	~10% market	Data/ML, Kubernetes expertise

Other players: Oracle Cloud, IBM Cloud, Alibaba Cloud, DigitalOcean...

Service Name Comparison

Category	AWS	Azure	GCP
Compute (VMs)	EC2	Virtual Machines	Compute Engine
Object Storage	S3	Blob Storage	Cloud Storage
Managed K8s	EKS	AKS	GKE
Serverless	Lambda	Functions	Cloud Functions
SQL Database	RDS	SQL Database	Cloud SQL
NoSQL	DynamoDB	Cosmos DB	Firestore/Bigtable

Same concepts, different names – skills transfer across providers

Why AWS for This Course?

- Market leader with broadest adoption (easiest to find jobs/resources)
- Most comprehensive free tier for learning
- Excellent documentation and tutorials
- Skills transfer easily to Azure/GCP (same concepts)
- HPC-specific offerings (ParallelCluster, HPC instances)

Key Takeaways (exam questions)

1. **NIST defines cloud through 5 characteristics**
   On-demand, broad access, pooling, elasticity, metering

2. **Service models trade control for convenience**
   IaaS → PaaS → FaaS → SaaS (more abstraction, less management)

3. **Cloud architecture and deployment**
   Composability, public/private/hybrid/multi cloud

4 .**Cloud converts CapEx to OpEx**
   Pay for use, not capacity; scale costs with business

5. **FinOps is essential for cloud success**
   Tag everything, set budgets, rightsize, clean up

References

Standards and Definitions:

- NIST SP 800-145: The NIST Definition of Cloud Computing
- FinOps Foundation: finops.org

Provider Documentation:

- AWS: aws.amazon.com/pricing, aws.amazon.com/economics
- Azure: azure.microsoft.com/pricing
- GCP: cloud.google.com/pricing

Cost Calculators:

- AWS Pricing Calculator: calculator.aws
- Azure Pricing Calculator: azure.microsoft.com/pricing/calculator

Questions?

Exercise: Cloud Economics Analysis

~2 hours

TCO analysis • Service model selection • Pricing comparison